

Introducing Agile Practices in a Doc-Driven Software Dev. Practice

## INTRODUCING AGILE PRACTICES IN A DOCUMENTATION-DRIVEN SOFTWARE DEVELOPMENT PRACTICE: A CASE STUDY

Lise Tordrup Heeager  
Aalborg University, Denmark  
Email: liseh@cs.aau.dk

### ABSTRACT

*The agile and the documentation-driven software methods advocate two different ways of developing software; but research does show some compatibility between these. Practitioners have an increasing desire to adopt agile software development methods while still being compliant with a quality assurance standard. However, due to little empirical research documenting software practices that meshes the agile and the documentation-driven methods, it is not well understood how to do this. This paper presents a case study of a software development practice in a pharmaceutical organization. For market reasons the organization had to comply with the US Food and Drug Administration standard. At the same time significant parts of the agile method Scrum had been implemented by the software group. The purpose of this paper was to understand the possibility of implementing a software development practice that meshes the agile and the documentation-driven methods and the challenges of doing this. This case study supports the previous literature and shows that it is possible to implement a software development practice, that meshes the agile and the documentation-driven methods. But, it also showed that doing so is not straight forward. It showed that meshing the project characteristics: 'developers', 'requirements', 'testing' and 'customer-relations' was especially difficult.*

**Keywords:** Agile Software Development, Documentation-Driven Software Development, Quality Standards, Scrum, FDA

### INTRODUCTION

In information systems development, an increasing amount of practitioners are interested in adopting methods (1), and the matter of choosing the appropriate software method is occupying many IT practitioners (2). Within the development of information systems, the agile and the documentation-driven methods advocate two different approaches to software development (3). The agile software methods (e.g. XP and Scrum) promise to deliver software without excessive cost (4,5). They are based on the assumptions that programs and programming is primary, not documentation. Agile development seeks to increase software quality by adhering to a very flexible strategy (6). The documentation-driven software methods are most often associated with the Waterfall model, but are also represented by quality standards, for example the ISO standards (7) and maturity models such as the CMMI (8). They rely on the assumption that to achieve high quality, software development processes must be defined and documented. The requirements are established and documented before designing, designs are established and documented before programming, testing is thorough and documented, and there are procedures through which documentation can be changed.

Neither of the agile nor the documentation-driven methods provide the ultimate approach, both has strengths and weaknesses (9). The documentation-driven methods work well for stable projects, but they lack the ability of handling change. The agile methods embrace change (4), but lack the ability to handle high complexity (10). Software organizations have therefore become increasingly interested in meshing the agile and the documentation-driven methods (11), in order to create a software practice, that takes advantage of the strengths and compensate for the weaknesses of the two (12). Even though the agile and documentation-driven methods seem contradicting, several researchers agree that they are compatible (13-16). At this point only little empirical research documents how to implement a meshed practice (17,18). Due to the differences between the methods, meshing the two is not straightforward and it is not well understood how to do this (19,20) - especially not when developing safety-critical software (21). When developing safety-critical software the software practice and the product have to comply with strict quality and process standards, that govern how the software should be developed. Implementing elements of the agile methods therefore become very challenging.

To fill this knowledge gap, this paper provides a detailed understanding of how a software group has implemented and improved a software practice meshing the agile and the documentation-driven methods. The purpose was to understand the possibility of implementing a software development practice, that meshes the agile and the documentation-driven methods and the challenges of doing so. This paper is based on a longitudinal, interpretive case study of a software development practice used by a software group in a large, pharmaceutical organization. The software group is developing a clinical, highly safety-critical product with embedded software, that by law has to comply with quality- and process standards such as the US Food and Drug Administration (FDA) standard. But, at the same time the software group has introduced several elements of the agile method, Scrum. The analysis gives an overview of the changes and improvement of the software practice identified at the two different times of data collection.

In the following section the theoretical background is presented and the analysis framework developed. In the third section the research method is outlined, followed by a description of the case. Then the analysis and evaluation of the software development practice is presented. Finally the results are discussed and concluded on.

## **THEORETICAL BACKGROUND**

This section presents the theoretical background for creating the analysis framework.

### **The Agile and the Documentation-Driven Methods**

The agile methods were developed based on the shortcoming and failures of documentation-driven methods. The agile manifest (developed in 2011) expresses some of the differences between the agile and the documentation-driven methods. The agile methods value: 1) individuals and interactions over processes and tools, 2) working software over extensive documentation 3) customer collaboration over contract negotiation and 4) responding to change over following a plan (22). Agile methods advocate an iterative, test-driven software development strategy with frequent customer feedback. Short iterations, multi-disciplinary teams, knowledge sharing and continuous integration allow control over the project and increase visibility (23). Agile methods

also advocate an informative workspace, in which the development is supported by information radiators - the placement of relevant information where passersby can see them (24). Refactoring (redesigning and rewriting software) is also advocated by the agile methods, as it serves to correct for poorly written and redundant code (21). eXtreme Programming (XP) (4) and Scrum (5) are the two most well-known agile methods. While XP offers an engineering perspective in terms of a range of practices and principles; Scrum provides guidance for efficient management of projects (25). Scrum operates with three roles and several practices (see table 1).

| Role/practice               | Description                                                                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Scrum master                | The Scrum Master is responsible for the scrum practice.                                                                                                 |
| Product owner               | The product owner is the customer.                                                                                                                      |
| Scrum team                  | Self-organizing project teams.                                                                                                                          |
| Sprints                     | The team works for a fixed period of time called a sprint (30 days is recommended).                                                                     |
| Daily scrum meeting         | Each scrum team meets daily for a 15-minutes status meeting; these meetings are often done standing up.                                                 |
| Sprint planning meeting/day | The next sprint goal and functionality are determined at the sprint planning meeting. The scrum team then devises the individual tasks to be performed. |
| Sprint review meeting       | The scrum team presents the product increment built during the sprint.                                                                                  |
| Product backlog             | A prioritized list of all product requirements.                                                                                                         |
| Release backlog             | The subset of the product backlog selected for a release.                                                                                               |
| Sprint backlog              | The tasks devised for a sprint.                                                                                                                         |
| Scrum boards                | Boards displaying the tasks chosen for the current iteration and their status.                                                                          |

Table 1. Roles and Practices of Scrum (5)

The documentation-driven methods strive at creating predictability, repeatability and optimization; the methods aim at achieving this by focusing on long-term plans controlled by long-term milestones (26). They rely on a documentation strategy in order to ensure and prove the quality of the product; in addition to the end product, documentation of different aspects of the software development and the product is produced (27). The project is controlled through a command-and-control style of management, including a clear separation of roles (28). The customer is included in the beginning of the project, where the requirements are specified (27). Documentation-driven methods are often associated with the Waterfall model (consisting of sequential development stages), quality standards and maturity models, such as the ISO9000 family of standards (7) and CMMI (8). For safety-critical software development even stricter quality standards must be applied; one example is the FDA standard. FDA approval relies on a process-driven approach. The manufacturers of the software obtain approval by showing that they have applied quality assurance techniques to certain levels of coverage (29). The development practice include: 1) identification of the system hazards and a definition of safety requirements, 2) a determination of how various components in the system can contribute to these hazards, 3) defining derived safety requirements for the components, and 4) the development of the components to meet these safety requirements (30).

The literature shows that the agile and the documentation-driven methods have some differences in their focus, in their customer-relations, in the amount of documentation, in their development

strategy and in the way they handle requirements. People-issues are at the heart of the agile methods (12) as they focus on the social aspects of software development (31). It is a significant departure from the focus on processes suggested by the documentation-driven methods (20). Both the documentation-driven and the agile methods highly value the satisfaction of the customer. But, in documentation-driven methods the customer is mainly involved during the early phase of the project, while agile methods involve the customer throughout the whole development (32). The handling of requirements is also a difference between the agile and the documentation-driven methods. While the agile methods base the requirements on user stories created and constantly updated by the customer (4), the documentation-driven methods advocate that the requirements are established and documented before designing and so forth (33). The amount of documentation is one of the most significant differences between agile and documentation-driven methods. Even though agile methods do not cast all documentation aside and it is not necessary to write piles of documentation in order to follow a documentation-driven method; agile methods do not support the degree of documentation demanded by documentation-driven methods (32,34). Agile methods advocate an iterative software development strategy (35), which opposed to the documentation-driven methods (which have separate coding and testing phases) include the testing and debugging of the written code in each iteration (36).

### **Compatibility of the Agile and the Documentation-Driven Methods**

Previous research agrees that the agile and the documentation-driven methods despite the differences to some degree are compatible, but disagree about the level of compatibility. Based on a literature study Heeager (37) concludes that it is possible to implement several agile elements in a software development practice and at the same time comply with a quality assurance standard. No evidence, however, shows that it is possible to implement a full agile software development practice in a documentation-driven setting.

Some researchers argue that the documentation-driven methods support the agile methods by providing a disciplined framework; CMMI for example presents a framework for process improvement with guidelines of what to do to improve (38). The documentation-driven methods can furthermore help identify the shortcomings of the agile methods by checking the coverage of each process area; the potential for improvement can thereby be identified (39). The ISO standard has been used to provide structure to a software practice and help ensure the agile method was followed (40). On the other hand agile practices ensure that processes are implemented efficiently while embracing change. The agile method contributes with adaptability, while the documentation-driven method contributes with discipline (25,41).

Both the agile (42) and the documentation-driven (13) methods are highly flexible and adaptable. Several researchers agree that agile methods can be adapted to ensure compatibility with standards, such as the ISO. Wright (43), for example, concludes that companies using XP can meet the requirements of standards such as ISO9001. The key is to base the quality management on the natural outputs of the development practice and not rely on work products produced for the sake of the quality assurance. Research also compares the agile methods and CMMI. There is a consensus that CMMI level 2 is largely compatible with the agile methods, whereas CMMI level 3 and agile methods are partially compatible. Comparing the process areas of CMMI with the agile methods, Alegria and Basticca (44) find that the process areas of CMMI level 2 is 72%

fulfilled, whereas the CMMI level 3 is only 60% fulfilled. XP and CMMI level 2 are highly compatible, because both XP and CMMI focus on basic project management; but an extension of XP is however needed to fulfil all requirements (45). XP fulfils many of the CMMI level 3 requirements as well; XP for example includes mechanisms for verification and criteria for completion (46). Comparing the process areas of CMMI level 4 and 5 and agile methods, they seem less compatible; XP needs management and infrastructure (47). A few case studies, however, show that it is possible to comply with CMMI level 4 or 5 (41,48). While the agile method ensures that processes are implemented efficiently, CMMI ensures that all relevant processes are considered (41).

### **The Home Grounds of the Agile and the Documentation-Driven Methods**

The home ground concept was introduced by Boehm (26); a home ground is project characteristics within which a method performs very well. As the home grounds compare the agile and the documentation-driven software methods on general project characteristics, they are useful for analyzing a specific software practice in order to determine if and how it meshes the agile and the documentation-driven methods. This paper will therefore adopt the home grounds as the analysis framework (table 2 and table 3). The descriptions of each characteristic (referred to as a category) and its sub category is based on the literature on the agile and the documentation-driven methods. Both the agile and the documentation-driven home grounds consist of 12 characteristics divided into 4 categories: personnel, management, technical and application (49).

| Construct            | Category                    | Subcategory                               | Description                                                                                                                                                        |
|----------------------|-----------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Documentation-driven | Personnel Characteristics   | Documentation-driven developers           | Documentation-driven methods focus on planning the project and defining the architecture of the software so that less-capable people can contribute with low risk. |
|                      |                             | Documentation-driven customers            | Documentation-driven methods rely on contractual plans and specifications written and agreed on in the beginning.                                                  |
|                      |                             | Documentation-driven culture              | The people feel comfortable and empowered when there are policies and procedures that define their role in the enterprise; each person's tasks are well-defined.   |
|                      | Management Characteristics  | Documentation-driven customer relations   | Documentation-driven methods depend on a contract between the developers and customers as the basis for customer relations.                                        |
|                      |                             | Documentation-driven planning and control | Documentation-driven methods rely on documented plans and a project manager in charge of the control.                                                              |
|                      |                             | Documentation-driven communications       | Documentation-driven methods rely heavily on explicit documented knowledge.                                                                                        |
|                      | Technical Characteristics   | Documentation-driven requirements         | Documentation-driven methods use complete, consistent, traceable, and testable specifications.                                                                     |
|                      |                             | Documentation-driven development          | Documentation-driven methods rely on extensive design.                                                                                                             |
|                      |                             | Documentation-driven testing              | Documentation-driven methods use late-testing based on documented test plans.                                                                                      |
|                      | Application Characteristics | Documentation-driven size                 | Documentation-driven projects scale best to large projects (Large teams and products).                                                                             |
|                      |                             | Documentation-driven primary objective    | The primary goals of documentation-driven methods are predictability, stability, and high assurance.                                                               |
|                      |                             | Documentation-driven environment          | Documentation-driven methods work best when the requirements are largely determinable in advance and remain stable.                                                |

Table 2: Analysis framework (documentation-driven characteristics), adapted from Boehm (26)

| Construct | Category                    | Subcategory                | Description                                                                                                                                                                 |
|-----------|-----------------------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Agile     | Personnel characteristics   | Agile developers           | Critical people-factors for agile methods include amicability, talent, skill, and communication.                                                                            |
|           |                             | Agile customers            | Agile methods strongly depend on dedicated, collocated customers.                                                                                                           |
|           |                             | Agile culture              | In an agile culture, the people feel comfortable and empowered when there is a high degree of freedom.                                                                      |
|           | Management characteristics  | Agile customer relations   | Agile developers use working software and customer participation to instil trust in their track record, the systems they have developed, and the expertise of their people. |
|           |                             | Agile planning and control | Agile methods see planning as a means to an end rather than a means of recording in text.                                                                                   |
|           |                             | Agile communications       | Agile methods rely heavily on tacit, interpersonal knowledge.                                                                                                               |
|           | Technical characteristics   | Agile requirements         | Most agile methods express requirements in terms of adjustable, informal stories.                                                                                           |
|           |                             | Agile development          | Agile methods advocate simple design, one that emerges as functionality is implemented.                                                                                     |
|           |                             | Agile testing              | Agile methods use executable test cases, which define requirements, the development is test driven.                                                                         |
|           | Application characteristics | Agile size                 | Agile methods seem to work best with small to medium teams of people working on relatively small applications.                                                              |
|           |                             | Agile primary objective    | The primary goals of agile methods are rapid value and responsiveness to change.                                                                                            |
|           |                             | Agile environment          | Agile methods are most applicable to turbulent, high change environments.                                                                                                   |

Table 3: Analysis framework (agile characteristics), adapted from Boehm (26)

## RESEARCH APPROACH

This is a longitudinal, interpretive single case study (51) of how a safety-critical product with embedded software is developed at a pharmaceutical company. The purpose of the study was to evaluate the software development practice used by the software group in one of their projects. To be able to identify changes in and improvements of the software practice, the empirical data was collected and validated iteratively through two phases (Table 4). Each phase consisted of 3 sub phases in order to introduce an iterative approach of data collection, analysis and validation. The data collection stretched over a period of approximately 2 years.

| Phase | Sub phase | Duration                   | Data collection                                                           | Data documentation                        |
|-------|-----------|----------------------------|---------------------------------------------------------------------------|-------------------------------------------|
| 1     | a         | November 2008 – June 2009  | 7 qualitative interviews based on an interview guide                      | Interviews audio recorded and transcribed |
|       | b         |                            | 8 (4+4) qualitative interviews based on 2 interview guides                | Interviews audio recorded and transcribed |
|       | c         |                            | Seminar presentation and validation with 11 members of the software group | Meeting audio recorded                    |
| 2     | a         | January 2010 – August 2010 | 7 qualitative interviews based on an interview guide                      | Interviews audio recorded and transcribed |
|       | b         |                            | 5 qualitative interviews based on an interview guide                      | Interviews audio recorded and transcribed |
|       | c         |                            | Observations of a planning meeting and a stand-up meeting                 | Meetings audio recorded and transcribed   |

Table 4: Phases in the interpretive research approach

The primary data collection technique was interviews; in qualitative research this is an important data gathering tool (50). It is through the interviews, the researcher has the best opportunity to access the interviewee's interpretations of the actions and events, which have or are taking place (51). The interviewees were chosen in order to include managers, developers and testers. This way each sub team of the software group was represented. A triangulation of the data was made as the interviews were supplemented by observations and document studies.

#### Data Collection: Phase 1

The first phase included two sub phases of interviews (1a + 1b) and a sub phase with a seminar presentation and validation of the results so far (1c). All interviews were organized as diagnostic interviews (52), based on a semi-structured interview guide. In the first sub phase (1a) 7 qualitative interviews were conducted with: the project manager, the software architect, a software tester and four developers from both sub teams. The interview guide included the subjects: FDA requirements, agile software development, the requirement specifications, handling of errors and general strengths and weaknesses. The second sub phase (1b) included eight interviewees: the project manager, the software architect, a tester and five developers from both sub teams. Four of these interviewees participated in the first phase, whereas the last four participated for the first time. Two interview guides were used. Interviewees participating for the second time were asked questions based on the activities identified in sub phase 1a and concluded by validating some of the statements from the first interviews. New interviewees were asked questions which included the subjects of FDA requirements and agile development before discussing the activities and evaluating the statements for the validation of the first phase. The findings were written into a report and presented to the software group in a seminar including a presentation and a discussion of these. The interviewees from the previous two sub phases participated.

### **Data Collection: Phase 2**

The second phase also included two sub phases (2a + 2b) of interviews and added a sub phase (2c) of observations in order to verify the procedure and outcomes of the planning meetings and stand-up meetings. The data collection was based on a framework of the home grounds of the agile and the documentation-driven methods (26). The interviews were semi-structured by an interview guide based on the characteristics of the home grounds. The purpose was to follow up on the evaluation from phase 1 and to identify changes in and improvements of the software practice. The first sub iteration (2a) included seven interviews with: 3 software developers, a tester, the new software architect, a coordinator between the software group and other groups of the project and a consultant specializing in Scrum, who had been affiliated to the software group for 5 months. The output of the interviews was an identification and description of activities within the characteristics of the home grounds and a revised evaluation. Sub iteration 2b focused on gaining a detailed description of the activities identified in sub iteration 2a. It included 5 interviewees: the former project manager, who was reassigned as responsible for the process, the former architect, now functioning as a software developer, the new scrum master, a tester and a software developer. The purpose of sub iteration 2c was to determine the procedure and the outcome of the planning meetings and stand up meetings. It included 2 observations, 1 observation of a planning meeting and 1 observation of a stand up meeting.

### **The Data Analysis**

In total twenty eight interviews, observations of meetings and document studies were conducted. All interviews were audio recorded and transcribed. As coding of the data provides a structured analysis process, the interviews were collected into one hermeneutic unit and coded using ATLAS.ti V6 (53). The coding was based on the framework of the home grounds of the agile and the documentation-driven methods, including the characteristics: agile or documentation-driven personnel characteristics, agile or documentation-driven management characteristics, agile or documentation-driven technical characteristics and agile or documentation-driven application characteristics. The codes consisted of the subcategories of the frameworks, making it easy to evaluate the software practice on each project characteristic. The full analysis were written up, presented in a report and validated by the software group.

### **CASE DESCRIPTION**

This is a study of a software group developing an embedded, clinical device for dispensing drugs. Due to the high level of safety-criticality, the overall device project had to be compliant with a large number of quality standards (ex. FDA), some applying directly to the software development practice. To obtain approval by the FDA one has to show that several quality techniques (associated with the documentation-driven methods) have been applied (29). Extensive documentation on how the quality has been secured must be written and approved. The development must therefore for example include: an identification and definition of system hazards and safety requirements (30).

The project studied was the first and by far the largest and most complicated software project in the organization. The study focuses on the software development practice. In the beginning, the software development suffered from high complexity, uncertainty of the tasks and shifting requirements. This led to a decision of implementing an agile method in the software group. The

decision was made internally in the software group and did not involve the rest of the project. The decision was mainly taken by the software project manager; who was later assigned the role as process manager. Scrum was chosen, as it at that point seemed simple and easy to implement. The purpose of implementing Scrum was to increase adaptability in order to cope with the uncertainties of the project. This is an analysis of how the software development practice has gone from being primarily documentation-driven to a practice that meshes elements of the documentation-driven FDA standard and elements of the agile software method, Scrum. During the period of the case study the software development group expanded from 17 to approximately 30 developers, testers and managers. At the end of the study, the software group consisted of the software project manager, a software architect, the process manager, a coordinator responsible for the coordination with the other parts of the project, the developers and the testers. The software project manager was the highest authority of the software group; this role shifted as the software group grew larger. At the beginning of the project the process manager was also the software project manager. He was very interested in agile software development, whereas the latest software project manager had a background in production and was much more traditional in his way of thinking.

The developers were divided into two sub teams, which were coordinated by the scrum master. The testers were organized in their own sub team controlled by a test manager. The two sub teams of developers worked on the same product backlog. The tasks of the iteration were displayed on two scrum boards on the back wall in the office. The developers primarily coordinated within the sub teams through the daily stand-up meetings. The developers were responsible for unit testing and reviewing the code during each iteration. The test team conducted the integration tests before the code was handed to the system engineering group that ran system tests of the increments, released every second month. The software development practice was divided into iterations which had varied in length but settled on 2 weeks. Each iteration was initiated with a planning meeting, that included a retrospective of the previous iteration and the planning of the future iteration. Even though both the developers and the testers ran iterations, the practice of the test team was very different from the practice of the developers. The testers used a highly documentation-driven development strategy that was managed by the test manager. Each member had an area of which they were responsible. For example: one was responsible for the design of the test cases while another was responsible for implementing the code, and a third responsible for automating the tests.

The product development also involved several other specialized groups: pharmaceutical and clinical researchers responsible for the drugs, process engineers responsible for production facilities, mechanical engineers responsible for the products mechanical functions dispensing the drugs, embedded hardware engineers responsible for the computer chips and software developers responsible for the software controlling the hardware. In total over 100 managers, researchers, engineers and developers were working on the device project. The software group was working in close cooperation and was dependent of the engineers developing the hardware, the mechanics and the system engineering group. Prior to this project, the organization had very little experience with software development and all software processes were to be developed from scratch. At the time of study, the project had been running for several years and was entering the last state of refining the product.

## ANALYSIS

The software development practice is analyzed according to the characteristics of the home grounds. The analysis focuses on the changes in and improvements of the software practice. Hence, in the analysis the software practice is depicted at two different timestamps: 1) at the time of the first data collection (year 2008-2009), referred to as the prior software practice and 2) at the time of the second data collection (year 2010), referred to as the present software practice. During the period of study, the software development practice went from being primarily documentation-driven to include several agile elements. The software group did still face several problems both due to internal issues in the software group and external issues caused by the project structure. The findings are summarized in table 5 (the prior software practice) and table 6 (the recent software practice). The most relevant characteristics are described in further details below.

### Developers

At the beginning, the organization had very little experience in software development projects. The majority of the software developers initially assigned to the project had a background in production and documentation-driven software development. Many of these developers were reluctant towards the adoption of Scrum, and had problems adjusting to the agile mindset. Following quote shows how a software developer expresses this reluctance towards agile software development at the first data collection (in the prior software practice): *"I think it has some damaging consequences; many others in the group think the same"* (a software developer, January 2009). Acknowledging that the competences of these software developers were ill suited for the project and for the ambition of implementing an agile software practice; the software developers hired during the project period had a strong software background and experience with agile software development. Additionally, several resources were spent on educating the developers, for example: a consultant facilitating the scrum practice was affiliated to the software group for six months. The final consequence taken by management of the software group was to remove some of the inexperienced developers from the development practice, assigning them to documentation related tasks. All of these initiatives had at the time of the recent software practice resulted in a more experienced software group, consisting of both agile and documentation-driven developers.

| Characteristics                      | Agile                                                                              | Meshed                                                                                                             | Documentation-driven                                                                        |
|--------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Developers                           |                                                                                    |                                                                                                                    | Primarily documentation-driven.                                                             |
| Customers                            |                                                                                    |                                                                                                                    | No customers affiliated.                                                                    |
| Culture                              |                                                                                    |                                                                                                                    | Primarily documentation-driven.                                                             |
| Customer relations                   |                                                                                    |                                                                                                                    | No relations or contact to a customer.                                                      |
| Planning and control                 |                                                                                    | Iterations of 4 weeks length, which often did not include testing and was sometimes disrupted to ensure stability. |                                                                                             |
| Communications and knowledge sharing | The communication of the developers was primarily face-to-face.                    |                                                                                                                    |                                                                                             |
| Requirements                         |                                                                                    |                                                                                                                    | The requirements were established in the beginning of the project and written in documents. |
| Development                          |                                                                                    |                                                                                                                    | The development was regulated by formal requirements.                                       |
| Testing                              |                                                                                    |                                                                                                                    | Much unit test and integration testing was postponed.                                       |
| Size                                 |                                                                                    |                                                                                                                    | A large software group                                                                      |
| Primary objective                    |                                                                                    |                                                                                                                    | The primary objective of the developers was being approved by FDA.                          |
| Environment                          | The software group had created an agile environment with for example scrum boards. |                                                                                                                    |                                                                                             |

Table 5: Summary of the prior software development practice

| Characteristics                      | Agile                                                                              | Meshed                                                             | Documentation-driven                                               |
|--------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------|
| Developers                           |                                                                                    | Both documentation-driven and agile developers.                    |                                                                    |
| Customers                            |                                                                                    |                                                                    | No customers affiliated.                                           |
| Culture                              |                                                                                    | The culture had both agile and documentation-driven aspects.       |                                                                    |
| Customer relations                   |                                                                                    | The product owner role was filled by the software project manager. | No relations or contact to a customer.                             |
| Planning and control                 | The software development was highly iterative, using iterations of 2 weeks length. |                                                                    |                                                                    |
| Communications and knowledge sharing | The communication of the developers was primarily face-to-face.                    |                                                                    |                                                                    |
| Requirements                         |                                                                                    | The requirements specifications were rewritten.                    |                                                                    |
| Development                          |                                                                                    |                                                                    | The development was regulated by formal requirements.              |
| Testing                              |                                                                                    |                                                                    | Much testing still had to be done.                                 |
| Size                                 | The developers were divided into sub teams.                                        |                                                                    |                                                                    |
| Primary objective                    |                                                                                    |                                                                    | The primary objective of the developers was being approved by FDA. |
| Environment                          | The software group had created an agile environment.                               |                                                                    |                                                                    |

Table 6: Summary of the recent software development practice

### Customer Relations

As the device was developed for a market and the end-users were not associated to the project, the software group had no relations or contact to a customer, neither in the prior software practice nor in the recent software practice. The relation to the end-user was a market analysis done in the beginning of the project by the marketing group. This made the customer-relations documentation-driven and made it difficult to fill the role of the product owner (advocated by Scrum). One solution would have been to assign the product owner role to a person in the project with a full overview of the requirements of the device (for example a person from the marketing group). In order to do that the software group tried to convince the management that a detached product owner was needed, but without any luck. Instead, the software group had (in the recent software practice) split the role and assigned the role as official product owner to the software

project manager and the role as technical product owner to the software architect. Neither the software project manager nor the software architect, however, had an adequate overview of the product and its requirements to take the decisions expected of a product owner. It had furthermore unfortunate complications to have a product owner who was also designing and implementing the system; no one saw the product with the eyes of the customer. In following quote the software architect expresses his concern with this missing product owner role: *"I am completely entangled in the software. I have to design the software - so if I have misunderstood something when I write the user stories, then it will be implemented in the software, guaranteed"* (The software architect, February 2009). As the requirements (based on written project documents) only were processed by the software architect before entering the product backlog the risk that misunderstandings occurred and the product did not fit the needs of the end-user was heightened.

### Planning and Control

The purpose of introducing Scrum was to introduce adaptability in the software development practice in order to cope with the complexity, uncertainty and shifting requirements. Initially (in the prior software practice) the software group followed the recommendations of Scrum and implemented iterations of 4 weeks length. The developers who were reluctant to the idea of an agile practice were sceptical towards introducing iterations and had trouble breaking the tasks down into smaller bits, that could be implemented and tested within each iteration. Furthermore, while the software group tried to use iterative development, the project as a whole was controlled by long-term plans and milestones. Hence, the agile, iterative software development strategy was embedded in the overall documentation-driven project. The software group was influenced by these milestones, that determined the goals and activities for each period in the project and therefore highly influenced the priorities and focus. Following quote shows how one of the milestones affected the software development: *"at our previous milestone we had to have several documents ready, at the cost of the progress on the product"* (the software architect, June 2010). The iterations were also affected as interruptions from other project groups happened regularly, even though the scrum master tried to enforce the agile recommendation of not letting anything interrupt the developers in the middle of an iteration. Several of these interruptions had to be dealt with, as they were serious matters affecting the whole device. The amount of urgent interruptions several times forced the software group to break of the iteration concerned. Some of these interruptions were caused by severe errors in the existing software discovered at the system test run at a release and from changes demanded by other project groups working sequentially.

After having used iterative software development for a longer period and the number of agile developers had increased, several of the developers began to see the advantages of iterative development. Lead by the agile consultant, the developers decided to decrease the iteration length to 2 weeks (in the recent software practice). These short iterations had several advantages. It gave the developers a steady work rhythm and a dedicated focus, as they constantly were forced to focus on the highest prioritized tasks. The short iterations also forced the developers to break the tasks down into even smaller bits, which gave an even greater transparency of the content of the tasks. This had a positive influence on the developers' ability to estimate the tasks.

### Communications and Knowledge Sharing

The communication and knowledge sharing between the developers were primarily face to face, both in the prior and in the recent software practice. The developers coordinated daily at the stand up meetings; at these meetings the current tasks and the progress of the iteration were discussed. In the beginning of the project the developers were very sceptical towards the stand-up meetings. Before the software group was divided into sub teams, the whole group participated in the same meetings; due to the growing size of the group these became very time consuming and partly uninteresting. After the division of the software group and after the developers gained experience in conducting stand-up meetings, several developers did, however, find great value in these meetings as they ensured that all members of the sub team kept a shared focus and gave the developers a frequent overview of the progress of the iteration. The planning meetings and the retrospectives also functioned as forums for communication and knowledge sharing about the tasks and the iteration progress. Each sub team of developers had its own planning meetings and stand up meetings; the coordination between the teams was facilitated by the scrum master. An informative work space in which the scrum boards played a central role was also a primary communication channel. At the scrum boards the tasks of an iteration and their status were easily accessed. At a weekly status meeting the progress was communicated to the entire software group and its management. These meetings were supported by another board focusing on whether or not the development followed the overall plan. All of these meetings resulted in a close cooperation between the developers: *"We know each other very well, so even though the one you need to talk to is not in your own team, you still know who to go to"* (a software developer, May 2010). Due to the requirements of the FDA standard, the software group still did extensive documentation. But, the writing of documents played a smaller role in the knowledge sharing within the software group, as new documentation was rarely distributed and found uninteresting by most of the developers. The development had however still a great focus on documents as these were required for the approval of the product.

### Requirements

The requirements were established in the beginning of the project based on a market analysis. The requirements were written down and communicated through a hierarchy of documents. The higher the document was placed in the hierarchy, the harder it was to change the requirement. Requirements were also developed based on a hazard analysis which revealed the safety hazards of the device. The software requirements written in the software requirements specification were by the software group transformed into a product backlog with prioritized and estimated tasks. These tasks were by the developers referred to as user stories, but they were not written by users and were not in the form of user stories proposed by agile methods.

Problems with the requirements resulted in a rewriting of the documents in the late stages of development (at the time of the recent software practice). Several of the higher requirements specifications were filled with requirements which were not specified properly, with requirements which were too specific and with conflicting requirements. According to the process manager this showed that the organization had not been able to specify the product in the early stages: *"it is an indication that the project or the organization did not know what kind of product they wanted to develop"* (the software process manager, June 2010). The software group experienced difficulties interpreting and implementing the requirements. The software developers did not have the

authority or knowledge to sort out conflicting requirements or fill in the missing details. In the end, the process manager therefore pushed a rewriting of these documents through. This rewriting of the requirements moved the project characteristic on requirements towards a more meshed practice in which requirements are improved during the development, however not in an highly, iterative and agile way.

### Testing

Both in the prior and in the recent software practice, the software group tried to implement time-boxed iterations with a fully implemented and tested increment as output. But, a lot of the testing was postponed until later in the project, the increments were therefore not fully tested before they were moved to the system engineering group. One reason was a focus on functionality. The quality of many of the unit tests implemented was also too low. The lack of testing contributed to a low quality of the code, in terms of low stability and in several errors when increments were delivered to the system engineering group. The developers had not been trained in writing unit tests: *"when they decided to introduce unit tests, they did not give people the necessary competences to actually writing good unit tests"* (the software architect, February 2009). Errors detected at the system test (performed by the system engineering group) had to be handled much more formally than the errors detected at the unit tests and integration tests. The high amount of errors found at the system test therefore decreased the agility.

In the recent software practice the developers did increase the amount of unit tests done within an iteration and the quality of each unit test, as 1) the skills of the developers were heightened; 2) the developers experienced the benefits of unit testing and were motivated to increase the amount of tests (a positive circle, more tests breeds more tests) 3) the perceived value of the testing was increased, as the agile consultant had a focus on this issue. The focus on integration test also increased heavily, and the number of testers responsible for the integration test of the code increased intensely through the project, especially in the last period. However, as the recent test team worked on a big backlog of tests, they were not able to test each increment directly after the iteration. This created a big cliff between the developers and the testers, whose cooperation was almost nonexistent at the time of the recent software practice.

### DISCUSSION

In this section the software development practice is discussed according to the literature. Table 6 summarizes the compatibility of the agile and the documentation-driven methods for each project characteristic.

| Characteristic                      | Description                                                                                                                                                                                                                                                     |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Developers                          | Changing the mindsets of developers can be difficult; developers trained in the documentation-driven domain may have a hard time adjusting to the agile mindset.                                                                                                |
| Customer-relations                  | When developing software for a market, it is difficult to implement a customer-driven practice. Assigning the product owner role to an internal person with the right competences requires an acknowledgement from the management, that this role is important. |
| Planning and control                | Implementing a highly iterative software practice in a documentation-driven setting is possible and doing so can have several advantages. The milestones may however affect the content of the iterations.                                                      |
| Communication and knowledge sharing | Implementing a communication and knowledge sharing strategy that is primarily agile and personalized is possible, but documentation is still needed.                                                                                                            |
| Requirements                        | Handling the requirements in an agile, iterative manner is difficult when the development is affected by the milestones.                                                                                                                                        |
| Testing                             | Implementing a test-driven practice may be difficult even though both the agile and the documentation-driven methods focus on test.                                                                                                                             |

Table 4. The compatibility of the project characteristics

### Developers

Changing from a documentation-driven software development practice to an agile one requires a different mindset and different skills (23). Changing the mindsets of developers can, however, be difficult; developers trained in the documentation-driven domain may have a hard time adjusting to the agile mindset. As presented in the analysis, the developers were reluctant towards implementing Scrum. Based on an empirical study of seven organizations, Cohn and Ford (36) also experienced that even though most developers have an appropriate level of scepticism towards the adoption of agile practices, some developers do resist the change. Hiring experienced developers enhanced the implementation of the Scrum method, in the case study of this paper. The literature also highlights how competent people are needed to make the implementation happen. Little evidence is given, that agile methods will work in the absence of above-average people (27).

### Customer-Relations

The documentation-driven software methods only involve the customer in the beginning of the product when creating the requirements (49). In agile software development a close customer contact and a customer-driven practice is advocated, in order to ensure that the end-product meets the needs and expectations of the customer (49). Due to this heavy reliance on people, agile methods are hard to implement (54). This case study showed that for a software group developing software aimed for the market implementing a customer-driven practice can be very difficult. The software group experienced a need for a product owner to (re)define the requirements, but, creating a relation to the customer was in this case study impossible as no users were affiliated to the project. Murru et al (55) propose that filling the product owner role by an internal person is a way to solve this issue. In this case study, it was not possible as the management (driven by the quality standards and not by agile principles) did not prioritize this role. Instead this project characteristic had a strong documentation-driven character.

## Planning and Control

Previous literature suggests that the development strategies of the agile and the documentation-driven methods are incompatible (27). This case study, however, showed that it is possible to implement an agile software development strategy by embedding a highly iterative software practice in the linear development strategy of the project. Doing so can have several advantages. One advantage is the increase of adaptability. The purpose of agile software methods are to be adaptable, collaborative, delivery-driven, people oriented and customer-focused (35). Rising and Janoff (56) acknowledge the many advantages of iterative software development; they also found that breaking the tasks down (to fit within the iterations) into a series of manageable chunks increased the transparency of the content of the tasks and eased the estimation. In this case the iterations were however affected by the long-term milestones guiding the overall project. The activities demanded to pass the milestones, such as documenting the requirements and the software design in the early phases influenced the workflow and the focus of the iterations and decreased the agility. To solve this issue, Boehm and Turner (34) suggest that milestones should be realigned and redefined to better fit the iterative approach. As the software group in this case study did not have any influence on the milestones and the fact that the milestones also had to fit with the other groups of the project working sequentially, this was not possible.

## Communication and Knowledge Sharing

While the documentation-driven methods facilitate knowledge sharing primarily through documentation, the agile methods facilitate knowledge sharing through informal communications among team members (49,57). These two different strategies are in the knowledge management literature, referred to as personalization and codification (58). This case study showed that a primarily agile and personalized knowledge sharing strategy can be implemented in a documentation-driven setting. The personalization strategy was preferred by the developers and was the strategy most used in practice. They communicated primarily at the planning meetings, daily scrum meetings and through the information radiators. Documentation was, however, still needed, due to the requirements for safety-critical software development. According to the developers writing documents was a minor part of their work and their knowledge sharing, but it did however still play a central role as they for example had to document each peer review. Hence, the software group had to write documentation for the sole purpose of complying with the FDA standard. This supports the findings in previous research, which shows that the agile and the documentation-driven software development methods are limited in the compatibility when it comes to documentation (32,34).

## Requirements

In agile methods the requirements are gathered throughout the iterative lifecycle (24), whereas in documentation-driven development (such as the Waterfall model) the requirements are gathered and specified in the beginning of the project (33). The way the requirements are handled is in the literature identified as a major challenge to meshing the agile and the documentation-driven methods (37). This case study showed how handling the requirements in an agile and iterative manner can be difficult. The long-term milestones controlling the overall project may dictate that the requirements specifications have to be approved in the beginning of the project. The software group was not able to handle their requirements in a highly iterative manner, even though they

experienced problems with the requirements specified. But, they did push one rewriting of the requirements specifications through, in the late stages of the project. Changing the requirements, in the late stages of the project, is, however, very expensive and an iterative gathering of the requirements would therefore have been preferred (4). To deal with this issue a realignment of the milestones will be necessary.

### Testing

Agile software methods suggest that test cases are written before the software and that all parts of an increment are tested within the iteration. These executable test cases define the requirements, while documentation-driven methods rely on documented test plans and procedures (49). Boehm and Turner (34) suggest that implementing test driven development is a way to mesh the agile and the documentation-driven methods. They argue that both the agile and the documentation-driven methods focus on test. This case study, however, showed that implementing a test-driven practice in a documentation-driven setting can be difficult. The software group did not (despite several attempts) succeed in implementing agile testing in their software practice. Much testing was postponed until the late stages of the project and in order to comply with the quality standards, much effort was put on creating test plans and procedures. Postponing the tests resulted in releases of unstable increments to system tests, that returned many errors. Dealing with errors found at the system test run by the system engineering group implied a heavy authorization process, while errors found at tests within the software group could be handled in a much lighter way.

### CONCLUSION

This paper is a study of the software development practice of a software group in a pharmaceutical company developing a highly safety-critical software system for drug dispensing. The software and the device in which it was embedded were therefore to be compliant with several process standards, for example the FDA standard. The purpose of this paper was to understand the possibility of implementing a software development practice, that meshes the agile and the documentation-driven methods and the challenges of doing so. To do this, the analysis is based on a framework developed from the home grounds (26) of the agile and the documentation-driven methods.

The software development practice presented in this paper meshes the practices of the agile software development method Scrum with a documentation-driven method complying with the FDA requirements. The software group was agile as they ran time-boxed iterations of 2 weeks, including a planning meeting and a retrospective. The control of the tasks and the progress of the iterations were supported by the use of scrum boards. The developers were divided into small self-organizing teams; within these teams they coordinated through daily stand-up meetings, while the scrum master facilitated the coordination between the two teams. The software practice was however also documentation-driven as the requirements were fixed early in the project and there was no customer contact or validation of the interpretation of the requirements before implementation.

This case study supports the previous literature and shows that it is possible to implement a software development practice, that meshes the agile and the documentation-driven methods. But,

it also showed that doing so is not straight forward. It is possible to develop the software in iterations within a long-term plan consisting of long-term milestones; but these iterations may be affected by the milestones as these are guiding the focus of the project. It is possible and advantageous to implement an agile, personalized communication and knowledge strategy, but, it is necessary to write extensive documentation as well. The case study, however, showed that meshing the project characteristics: 'developers', 'requirements', 'testing' and 'customer-relations' were especially difficult. For documentation-driven developers it can be very hard to change to an agile mindset in which the tasks need to be broken down to fit in the iterations and in which knowledge sharing is primarily face-to-face. It is difficult to handle the software requirements in an iterative manner, as the long-term milestones controlling the project require an approval of these in the early stages of the project. Implementing a test-driven and a customer-driven practice also requires priority from management, for example implementing a functioning product owner role needs priority from management.

## REFERENCES

1. Avison, D.E. & Fitzgerald, G. (2003). Where now for development methodologies? *Communications of the ACM*, 46(1), 78-82.
2. Fitzgerald, B., Russo, N.L. & Stolterman, E. (2002). *Information systems development: Methods in action*. New York: McGraw-Hill.
3. Dahlberg, H., Ruiz, F.S. & Olsson, C.M. (2006). The role of extreme programming in a plan-driven organization. *The Transfer and Diffusion of Information Technology for Organizational Resilience: IFIP TC8 WG 8.6 International Working Conference*, 291-312.
4. Beck, K. & Andres, C. (2004). *Extreme programming explained: Embrace change*. Boston: Addison-Wesley Professional.
5. Schwaber, K. & Beedle, M. (2001). *Agile software development with Scrum*. New Jersey: Prentice Hall.
6. (6) Cockburn, A. & Highsmith, J. (2002). Agile software development, the people factor. *Computer*, 34(11), 131-133.
7. Hoyle, D. (2006). *ISO 9000 quality systems handbook*. Oxford: Butterworth-Heinemann.
8. Chrissis, M.B. & Konrad, M. & Shrum, S. (2003). *CMMI guidelines for process integration and product improvement*. Boston: Addison-Wesley Longman Publishing Co., Inc.
9. Boehm, B. & Turner, R. (2003). Observations on balancing discipline and agility. *Proceedings of the Agile Development Conference*, 32-39.
10. Dahlbom, B. & Mathiassen, L. (1993). *Computers in context: The philosophy and practice of systems design*. USA: Wiley-Blackwell.
11. Pikkariainen, M. & Mäntyniemi, A. (2006). An approach for using CMMI in agile software development assessments: experiences from three case studies. *SPICE 2006 conference*, 4-5.
12. Galal-Edeen, G.H., Riad, A.M. & Seyam, M.S. (2007). Agility versus discipline: Is reconciliation possible? *International Conference on Computer Engineering & Systems, ICCES'07*. 331-337.
13. Baker, S.W. (2005). Formalizing agility: An agile organization's journey toward CMMI accreditation. *Proceedings of the Agile Development Conference*, 185-192.
14. Heeager, L.T. & Nielsen, P.A. (2009). Agile Software Development and its Compatibility with a Document-Driven Approach? A Case Study. *Australasian Conference on Information Systems*, 205.
15. Manhart, P. & Schneider, K. (2004). Breaking the ice for agile development of embedded software: an industry experience report. *Proceedings of the 26th International Conference on Software Engineering*, 378-386.

16. Nawrocki, J., Olek, L., Jasinski, M., Paliswiat, B., Walter, B., Pietrzak, B. & Godek, P. (2006). Balancing agility and discipline with xprince. Rapid Integration of Software Engineering Techniques, Second International Workshop, RISE 2005, 266.
17. Kähkönen, T. & Abrahamsson, P. (2004). Achieving CMMI level 2 with enhanced extreme programming approach. Proceedings of the 5th International Conference, PROFES 2004, 378-392.
18. Pikkarainen, M. (2009). Towards a Better Understanding of CMMI and Agile Integration-Multiple Case Study of Four Companies. Product-Focused Software Process Improvement: 10th International Conference, PROFES 2009, 401-415.
19. Reifer, D.J. (2003). XP and the CMM. IEEE Software, 20(3), 14-15.
20. Vinekar, V., Slinkman, C.W. & Nerur, S. (2006). Can agile and traditional systems development approaches coexist? An ambidextrous view. Information Systems Management, 23(3), 31-42.
21. Vogel, D.A. (2006). Agile Methods: Most are not ready for prime time in medical device software design and development. DesignFax Online, 1-6.
22. Beck K, Beedle M, van Bennekum A, Cockburn A, Cunningham W, Fowler M, et al. Manifesto for Agile Software Development. 2001; Available at: <http://agilemanifesto.org>. Accessed October 9, 2011.
23. Mahanti, A. (2004). Challenges in enterprise adoption of agile methods - A survey. Journal of Computing and Information Technology, 14(3), 197-206.
24. Cockburn, A. (2006). Agile software development: The cooperative game. Boston, USA: Addison-Wesley Professional.
25. Jakobsen, C.R. & Johnson, K.A. (2008). Mature agile with a twist of CMMI. Agile, 2008. AGILE'08. Conference, 212-217.
26. Boehm, B. (2002). Get ready for agile methods, with care. Computer, 35(1), 64-69.
27. Nerur, S., Mahapatra, R.K. & Mangalaraj, G. (2005). Challenges of migrating to agile methodologies. Commun ACM, 48(5), 73-78.
28. Moe, N.B., Dingsøyr, T. & Dybå, T. (2010). A teamwork model for understanding an agile team: A case study of a Scrum project. Information and Software Technology, 52(5), 480-491.
29. Lee, I., Pappas, G.J., Cleaveland, R., Hatcliff, J., Krogh, B.H., Lee, P., Rubin, H. & Sha, L. (2006). High-confidence medical device software and systems. Computer, 39(4), 33-38.
30. Heimdahl, M.P.E. (2007). Safety and software intensive systems: Challenges old and new. Future of Software Engineering (FOSE '07), 137-152.
31. McAvoy, J. & Butler, T. (2009). A failure to learn in a software development team: The unsuccessful introduction of an agile method. Proceedings of the 16th International Conference on Information Systems Development (ISD2007), 1-13.
32. Paetsch, F., Eberlein, A. & Maurer, F. (2003). Requirements engineering and agile software development. Proceedings of the Twelfth International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises, 308.
33. Pressman, R.S. (2000). Software engineering - A practitioner's approach. UK: McGraw-Hill Publishing Company.
34. Boehm, B. & Turner, R. (2005). Management challenges to implementing agile processes in traditional development organizations. IEEE Software, 22(5), 30-39.
35. Larman, C. (2004). Agile and iterative development: a manager's guide. Boston, USA: Addison-Wesley Professional.
36. Cohn, M. & Ford, D. (2003). Introducing an agile process to an organization [software development]. Computer, 36(6), 74-78.
37. Heeager, L.T. (2012). The Agile and the Disciplined Software Approaches: Combinable or Just Compatible? Information Systems Development (ISD2011).
38. Elshafey, L.A. & Galal-Edeen, G.H. (2008). Combining CMMI and Agile Methods. Proceedings on the 6th International Conference on Informatics and Systems, 27-39.

39. Fritzsche, M. & Keil, P. (2007). Agile Methods and CMMI: Compatibility or Conflict? *e-Informatica Software Engineering Journal*, 1(1), 9-26.
40. McMichael, B. & Lombardi, M. (2007). ISO 9001 and agile development. *Proceedings of the AGILE 2007*, 262-265.
41. Sutherland, J., Jakobsen, C.R. & Johnson, K. (2007). Scrum and cmmi level 5: The magic potion for code warriors. *AGILE 2007*, 272-278.
42. Theunissen, W.H., Kourie, D.G. & Watson, B.W. (2003). Standards and agile software development. *Proceedings of the 2003 annual research conference of the South African institute of computer scientists and information technologists on Enablement through technology*, 178-188.
43. Wright, G. (2003). Achieving ISO 9001 certification for an XP company. *Extreme Programming and Agile Methods, Third XP Agile Universe Conference*, 43-50.
44. Alegria, J.A.H. & Bastarrica, M.C. (2006). Implementing CMMI using a Combination of Agile Methods. *CLEI electronic Journal*, 9(1), 1-15.
45. Bos, E. & Vriens, C. (2004). An agile CMM. *proceedings of 4th Conference on Extreme Programming and Agile Methods-XP/Agile Universe*, 129-138.
46. Laurila, P. (2004). Are CMM and agile methods really compatible? T-76.650 Seminar in software engineering, 1-10.
47. Paulk, M.C. (2001). Extreme programming from a CMM perspective. *IEEE Software*, 18(6), 19-26.
48. Anderson, D.J. (2005). Stretching agile to fit CMMI level 3. *Proceedings of the Agile Development Conference*, 193-201.
49. Boehm, B. & Turner, R. (2003). Using risk to balance agile and plan-driven methods. *Computer*, 36(6), 57-66.
50. Myers, M.D. & Newman, M. (2007). The qualitative interview in IS research: Examining the craft. *Information and Organization*, 17(1), 2-26.
51. Walsham, G. (1995). Interpretive case studies in IS research: nature and method. *European journal of information systems*, 4(2), 74-81.
52. Iversen, J.K., Nielsen, P.A. & Nørbjerg, J. (1998). Problem diagnosis software process improvement. *Information systems: Current Issues and Future Changes IFIP*, 30 66-81.
53. Muhr, T. (1991). ATLAS/ti—A prototype for the support of text interpretation. *Qualitative Sociology*, 14(4), 349-371.
54. Paulk, M.C. (2002). Agile methodologies and process discipline. *Crosstalk-The Journal of Defense Software Engineering*, 1(1), 15-18.
55. Murru, O., Deias, R. & Mugheddue, G. (2003). Assessing XP at a European internet company. *Software, IEEE*, 20(3), 37-43.
56. Rising, L. & Janoff, N.S. (2002). The Scrum software development process for small teams. *Software, IEEE*, 17(4), 26-32.
57. Crawford, B., Castro, C. & Monfroy, E. (2006). Knowledge management in different software development approaches. *Advances in Information Systems, LNCS 4243*, 304-313.
58. Hansen, M.T., Nohria, N. & Tierney, T. (1999). What's your strategy for managing knowledge? *Harv Bus Rev*, 77(2), 106-116.

## AUTHOR BIOGRAPHY

**Lise Tordrup Heeager** received the Master's degree in Informatics from Aalborg University, Aalborg, Denmark in 2008. She is currently working toward the Ph.D. degree at the Department of Computer Science, Aalborg University. Her current research interests include information systems and software engineering with a particular emphasis on meshing agile and documentation-driven methods.